Question 2

Not answered

Marked out of 1.00

Consider the following problem of counting the number of blue Easter eggs painted around each consecutive day.

Input: integer **t** > 4 (number of days), array **col**[1..**n**] (colors), and array **day**[1..**n**] (days).
There are n Easter eggs in total, and each day is an integer from 1 to t. The color of the i-th Easter egg (for i=1,...,n) is col[i], and it has been painted on the day day[i].
(There can be more than one Easter egg painted on the same day.)

Output: array **ret**[4..t] where each ret[p] is the total number of blue Easter eggs (that is, of color equal to **'blue'**) painted within one day before, after, and on day p; that is, within the days p-1, p, and p+1.

Your task:
Design **a slow and a fast** algorithm that solves this problem. Do it by dragging and dropping an appropriate code snippet on each line.

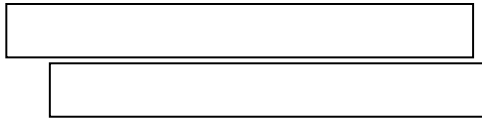
Note that *not all* code snippets will be used.

Slow algorithm:

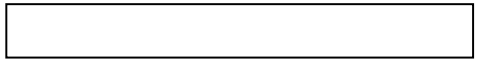
```
ret[4..t] = new array filled with 0s
for p=4 to t:
    [ ]
        [ ]
            if col[i] == 'blue':
                [ ]
return ret
```

Fast algorithm:

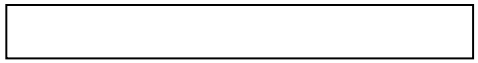
```
ret[4..t] = new array filled with 0s
[ ]
for j=1 to n:
```



for p=2 to t:



for p=4 to t:



return ret

The code snippets:

ret[p] = cum[p+1] - cum[p-2]	ret[p] = day[i]	cum[j] = day[j] + cum[j-1]
cum[p] += cum[p-1]	ret[p] = cum[p+1] - cum[p-1]	ret[p] = cum[p] - cum[p-2]
for i=1 to n:	ret[p] = cum[p] - cum[p-1]	cum[0..t] = new array filled with 0s
if col[j] == 'blue':	if p-1 < day[i] < p+1:	ret[p] += col[i]
ret[p] += 1	if p-1 <= day[i] <= p+1:	cum[day[j]] += 1

Question 3

Correct

Marked out of 1.25

Simplify the following running time as far as possible:

$$\Theta(n^{3+5} + 1000 \cdot \sqrt{n} + n^7 \cdot \log n)$$

Select one:

- $\Theta(2^n)$
- $\Theta(n^8)$ ✓
- $\Theta(1)$
- $\Theta(n^2)$
- $\Theta(n^7 \log n)$
- $\Theta(n^2 \log n)$
- $\Theta(\log n)$
- $\Theta(\sqrt{n})$
- $\Theta(n^6)$
- $\Theta(\sqrt{n} \log n)$
- $\Theta(n \log n)$
- $\Theta(n^3)$
- $\Theta(n^7)$
- $\Theta(n^4)$
- $\Theta(n)$

- $\Theta(n^8 \log n)$
- $\Theta(n^6 \log n)$
- $\Theta(n^5)$

Question 4

Correct

Marked out of 1.25

Simplify the following running time as far as possible:

$$\Theta(\log_3((9^n)^n) + 9 \cdot \log_3(n^n))$$

Select one:

- $\Theta(1)$
- $\Theta(\log_3 3)^n$
- $\Theta(2^n)$
- $\Theta(((81)^n)^n)$
- $\Theta(n^2 \log n)$
- $\Theta(\log n)$
- $\Theta(n^2)$ ✓
- $\Theta(n^4)$
- $\Theta(n^3)$
- $\Theta(n)$
- $\Theta(n^9)$
- $\Theta(n^5)$
- $\Theta(n \log n)$
- $\Theta(n^{10})$
- $\Theta(9^n)$

- $\Theta(3^n)$

Question 5

Correct

Marked out of 1.00

Select *all* correct statements.

Select one or more:

- a. $n^{20} = \Omega(n^{21})$
- b. $13\sqrt{n} = \Theta(\sqrt{n})$ ✓
- c. $n^{16} = O(n^{15})$
- d. $13n^{33} = O(n^{34})$ ✓
- e. $n^3 = \Theta(n^4)$
- f. $n^{26} = \Omega(n^{25})$ ✓

Question 6

Partially correct

Marked out of 0.75

There are three algorithms A, B and S. Let n be the input size. We only know that the running time

of algorithm A is $O(5^n)$,
of algorithm B is $\Theta(9^n)$, and
of algorithm S is $\Omega(7^n)$.

Select *all* correct statements:

Select one or more:

- [x] a. Algorithm A is faster than algorithm B. ✓
- [ ] b. Algorithm B is faster than algorithm A.
- [ ] c. We cannot tell whether algorithm A or algorithm B is faster.
- [x] d. Algorithm A is faster than algorithm S. ✓
- [ ] e. Algorithm S is faster than algorithm A.
- [ ] f. We cannot tell whether algorithm A or algorithm S is faster.
- [x] g. Algorithm B is faster than algorithm S. ✗
- [ ] h. Algorithm S is faster than algorithm B.
- [ ] i. We cannot tell whether algorithm B or algorithm S is faster.

Question 7

Partially correct

Marked out of 1.75

Consider the following algorithm *(the lines are numbered)*.

Input: array $a[1..n]$ (of integers).

Output: ?

```
1:  le = 0
2:  ri = min(3, n)
3:  count = 0
4:  while le < ri :
5:    if a[le+1] > 2:
6:      count = count + 1
7:    for i = 0 to 321 :
8:      count = count
9:    le = 1 + le
10: return count
```

1. What is the output of the algorithm above if the input is

$$a = \langle 3, 2, 3, 1, 0, 3 \rangle \, ?$$

Output: [ ] ⊗ *(Write a single number. Note that the first element of a is $a[1]$, the last one is $a[n]$.)*

2. What is the running time of the algorithm?

- $\Theta(2^n)$
- $\Theta(\log n)$
- $\Theta(\sqrt{n})$
- $\Theta(n \log n)$
- $\Theta(n^2 \log n)$
- $\Theta(n^3)$
- $\Theta((\log n)^2)$

- ◉ $\Theta(n^2)$ ⊗
- ○ $\Theta(1)$
- ○ $\Theta(n)$

3. For simplicity, we define $a[1..0]$ and $a[n+1..n]$ as empty subarrays of a. Select *all* invariants that (always) hold on line 4–-even if they are not helpful to prove the correctness:
   - ☐ $ri > 3$
   - ☑ $le \leq 3$ ◐
   - ☐ $count$ is the total number of elements in $a[1..le]$ with value at least '2'.
   - ☑ $count$ is the total number of elements in $a[1..le]$ with value at least '3'. ◐
   - ☐ $le \geq 1$
   - ☐ $count$ is the total number of elements in $a[ri..n]$ with value at least '2'.
   - ☐ $ri \leq n$
   - ☐ $count$ is the total number of elements in $a[ri..n]$ with value at least '3'.
4. Choose a line (by its line number) whose duplication will make the output twice as large.

   6 ✓

Question 8

Partially correct

Marked out of 1.00

Suppose we have an algorithm which, given an input x, proceeds as follows:

- *if x is of size at most 9, it computes the output in time $\Theta(1)$,*
- *if x is of size n with $n > 9$,*
  - *it first computes a temporary array in time $\Theta(n^4 \log n)$.*
  - *Then, it makes seven recursive calls of itself, each time on an input of size $\lceil n/2 \rceil$.*
  - *After the seven calls, it combines the seven results in total time $\Theta(n)$ and returns the output in time $\Theta(1)$.*

Give appropriate values for a and b, and select the correct function $f(n)$ such that the running time $T(n)$ of the algorithm above satisfies the recurrence $T(n) = a\,T(n/b) + \Theta(f(n))$.

$a =$ [ ] (*Write only a single number.*) $b =$ [ ] (*Write only a single number.*)

$f(n) =$

- $(\log n)^2$
- $(\log n)^7$
- $(\log n)^9$
- 1
- 2
- 7
- 9
- $\log n$
- n
- $n \log n$
- n^2
- $n^2 \log n$
- n^3
- $n^3 \log n$
- n^4
- $n^4 \log n$ ✓

- $\sqrt{n}$

*Note: You are **not asked to solve the recurrence!***

Question 9

Partially correct

Marked out of 0.75

Assign appropriate integer values to the variables X, Y and Z of algorithm AlgoD(integer n) such that its running time $T(n)$ satisfies the recurrence

$$T(n) = 16 \cdot T(\lfloor n/42 \rfloor) + \Theta(n^{38}) \ .$$

You don't need to understand what the algorithm computes!

(Write only single numbers.)

- *(Line 5)* X = 37
- *(Line 6)* Y = 47
- *(Line 7)* Z = 8

The division at line 14 is an integer division, that is, the result of n//Y is the largest integer not larger than n/Y.

```
1:  AlgoD(integer n):
2:    if n <= 1 :
3:      return 1
4:    else :
5:      X = ???
6:      Y = ???
7:      Z = ???

8:      ret = 0
9:      w = 1
10:     for j=1 to X :
```

```
11:         w = w*n
12:       for k=1 to w :
13:         ret = ret + n

14:       m = n//Y
15:       for i=1 to Z :
16:         ret += AlgoD(m)
17:         ret += AlgoD(m)

18:       return ret
```

Question 10

Incorrect

Marked out of 1.25

Solve the following recurrence using the master theorem. Determine (for yourself) the values for a, b, and $f(n)$ and write down the value for c (as defined in the master theorem). Then, determine which case holds and write the resulting running time. (For case C, you can assume that $f(n)$ satisfies the regularity condition.)

$$T(n) = 27\,T(n/3) + \Theta(n^3)$$

$c =$ ⊗ (*Write only a single number.*)

Which case of the master theorem holds?

- Case A
- Case B
- Case C

What is the resulting running time?

$T(n) =$

- $\Theta(n^5)$
- $\Theta(1)$
- $\Theta(n^7 \log n)$
- $\Theta(\log n)$
- $\Theta(n \log n)$
- $\Theta(n^7)$
- $\Theta(n^4 \log n)$
- $\Theta(n^3)$ ⊗
- $\Theta(n^2)$
- $\Theta(n^4)$
- $\Theta(n)$
- $\Theta(n^6)$
- $\Theta(n^2 \log n)$

- $\Theta(n^6 \log n)$
- $\Theta(n^3 \log n)$
- $\Theta(n^5 \log n)$

Question 11

Correct

Marked out of 1.00

Solve the following recurrence using the master theorem. Determine (for yourself) the values for a, b, and $f(n)$ and write down the value for c (as defined in the master theorem). Then, determine which case holds and write the resulting running time. (For case C, you can assume that $f(n)$ satisfies the regularity condition.)

$$T(n) = \Theta(n^4 \log n) + 10 \cdot 10^4 \cdot T(n/10)$$

$c =$ 5 ✓ (*Write only a single number.*)

Which case of the master theorem holds?

- (●) Case A ✓
- ( ) Case B
- ( ) Case C

What is the resulting running time?

$T(n) =$

- (●) $\Theta(n^5)$ ✓
- ( ) $\Theta(n^2)$
- ( ) $\Theta(n^7 \log n)$
- ( ) $\Theta(n)$
- ( ) $\Theta(n^2 \log n)$
- ( ) $\Theta(n^3)$
- ( ) $\Theta(n^6 \log n)$
- ( ) $\Theta(1)$
- ( ) $\Theta(n^4)$
- ( ) $\Theta(n^5 \log n)$
- ( ) $\Theta(n^4 \log n)$
- ( ) $\Theta(n^7)$
- ( ) $\Theta(n^6)$

- $\Theta(n^3 \log n)$
- $\Theta(n \log n)$
- $\Theta(\log n)$

Question 12

Partially correct

Marked out of 0.75

Consider the following array $a[1..18]$ containing eighteen elements:

$$a = \langle 110, 75, 90, 75, 55, 45, 80, 70, 20, 30, 55, 35, 45, 85, 30, 15, 65, 20 \rangle$$

Choose an appropriate index position j and **increase** the value of $a[j]$ such that a satisfies the heap invariant.
Note that the array starts **at index** 1. Thus, for example, $a[1] = 110$ and $a[18] = 20$.
Hint: Draw the array as a tree.

What index j do you choose? 6 ⊗ *(Write a single number between 1 and 18; there is only one correct answer.)*

What is the new value for $a[j]$? 56 ◐ *(Write a single number; the value should be larger than the old one; there is more than one correct answer.)*

Question 13

Not answered

Marked out of 1.00

Complete the following algorithm with exactly two code snippets from below such that

- the array *a* is sorted in a stable way,
- the running time is as small as possible, and
- the return value is n

Note that the for loop in the algorithm makes no sense, however, it is there to make the running time analysis more challenging.

Input: array *a*[**0**..***n-1***] of integers containing all values from 1 to *n*.
Output: *a* is sorted and the value *n* is returned..

Algorithm:
1. ret = 0
2. [ ]
3. for i=1 to (log n)*(log n):
4. [ ]
5. return ret

Drag and drop the appropriate code snippets on the respective lines. Exactly two code snippets must be used. Only one solution is correct. (The second argument in the sorting algorithms is the target search value.)

What is the running time of the correctly completed algorithm? Drag and drop the correct running time below.
Time: [ ]

heap_sort(a) | counting_sort(a) | quick_sort(a) | merge_sort(a) | insertion_sort(a)

ret = linear_search(a, n+1)

ret = binary_search(a, 0)

ret = linear_search(a, n-1)

ret = binary_search(a, n)

Theta(log n)

Theta(n (log n)²)

Theta((log n)³)

Theta(n log n)

Theta(n²)

Theta(n² log n)

Theta(n)

Previous activity

Exam: Registration

Jump to...

Next activity

1. Short Test